

Chef Vision 当前整体框架与后续优化思路

这是一份给自己看的整理稿：重点是看清现在项目到了哪、后面最值得先优化什么。

一、当前系统主链路



当前最重要的理解：这个项目已经不是“单一 RGB 算法”，而是 RGB 正向 + RGB 反向 + IR 温度线索共同工作的多模态框架。

所以后面优化时，不应该只盯某一个 mask，而应该看三条线之间：谁是主逻辑、谁负责兜底、谁提供约束。

二、现在的代码结构已经到了什么程度

这页的重点不是列文件名，而是帮你判断：哪些已经整理好了，哪些还应该谨慎对待。

已经基本拆开的层

入口与配置层

- track_config.py: 统一管理视频路径、温度文件、短跑参数、命令行入口。
- label_io.py: 统一读取和整理标注结果，主程序不再自己处理这些细节。

算法支撑层

- ir_wok.py: IR 锅区是独立模块了，后面换新方案有明确落脚点。
- rgb_forward.py / rgb_inverse.py: 两套 RGB 方案的辅助逻辑已经抽出去，不再全部堵在主循环里。

温度与投影层

- temp_fusion.py: 温度测量和 IR 食材聚类逻辑集中在这里。
- projection_utils.py: 专门放 RGB -> IR 的公共投影函数，只负责“使用矩阵”，不负责“计算矩阵”。

输出与表现层

- output_utils.py: Excel、曲线图、合成视频输出。
- viz_utils.py: overlay 和底部曲线这类可视化细节。

TrackFood.py 现在主要还剩什么

- 主追踪流程的总调度：什么时候读数据、什么时候进 SAM2、什么时候调用 IR/RGB 各模块。
- chunk 组织、视频帧处理、部分核心 helper。
- 少量还没继续外拆的核心算法函数。

当前的现实判断：工程结构已经基本够用了。后续继续硬拆的收益在下降，后面真正决定效果提升的，主要还是算法稳定性。

三、后续优化应该怎么排优先级

按“收益高低 + 现在项目状态”来排，不建议继续机械拆分。

建议的优化顺序

1. 先抓 IR 锅区这条主矛盾

- IR 锅区方案：这是当前最值得优先打磨的一块。结构已经准备好了，后面应该把注意力放到方案本身，而不是主程序怎么拆。
- 优先目标可以定为：第一帧人工给一个严格位于锅内的 IR mask，后续通过 IR 帧间配准估计平移，再整体平移锅区。
- 这样做的意义是：锅区约束先稳定下来，RGB 正向、RGB 反向和温度统计三条线都会跟着受益。

2. 再抓两个 RGB 方案的稳定性

- RGB 正向稳定性：重点不是换掉 SAM2，而是把“何时判定异常、何时引入 IR 约束、何时触发补强”梳理得更清楚。
- RGB 反向语义：继续保持“只在满足条件时才自动重标”，并保留触发图，方便回看触发是否过多或过少。
- IR 温度分割：后面要逐步把“锅底 / 食材 / 背景”的温度划分做成更可比较的方案，不然温度结果容易波动。

3. 最后补验证和少量工程修整

- 验证策略：后面每次改算法，尽量都用同一批测试视频短跑 + 完整跑，方便横向比效果，而不是只靠主观感觉。
- 工程上还能做的小优化有，但不建议优先。比如主循环再细拆一层编排函数，这种事情可以放在算法稳定之后再做。

一句话总结：现在最值得做的不是继续拆文件，而是把 IR 锅区方案先稳定下来，再带动 RGB 正向、RGB 反向和温度统计一起变稳。